

PopScript

Language Reference & Documentation

Version 1.0.4

pop-inc.github.io

Table of Contents

1. Overview
2. File Structure
3. Comments
4. Types
5. Variables
6. Operators
7. Control Flow — if
8. Loops — when, for
9. Functions
10. Built-in Functions
11. Libraries
12. Built-in Libraries — random, file, time, ui
13. Memory Management
14. CLI — pop run, build, get, list
15. Examples

1. Overview

PopScript is a statically typed interpreted language designed around explicit syntax, readability, and deterministic parsing. Every line reads like a sentence. There are no implicit conversions, no magic behaviors, and no ambiguous syntax.

PopScript source files use the `.pscript` extension. Library files use `.plib`. The toolchain is called `pop`.

Design principles

Static typing	All types are declared explicitly and resolved at parse time.
Explicit syntax	No implicit behavior. Every operation is visible in the code.
Readable aloud	for fruit with fruits; reads as plain English.
Deterministic parsing	One way to write each construct. No dual syntax.
Single function call syntax	All calls use parentheses: <code>func(arg)</code> .

2. File Structure

A PopScript file has no required boilerplate. Code runs top to bottom. The optional `armemory` directive must appear on the very first non-empty line if used.

```
$/ optional memory mode directive
armemory=false

$/ imports
lib import random

$/ variable declarations
int x = 10

$/ function declarations
func greet(name);
    print("Hello" name)
stop;

$/ main code
greet("world")
```

3. Comments

Comments start with `$/` and run to the end of the line. There are no multi-line comments.

```
$/ this is a comment
int x = 5  $/ inline comment
$/ print(x)  this line is ignored
```

4. Types

PopScript has five built-in types. All variables must be declared with an explicit type.

Keyword	Description
<code>int</code>	64-bit signed integer. Example: 42, -7, 0
<code>float</code>	64-bit floating point. Example: 3.14, -0.5
<code>string</code>	UTF-8 text in double quotes. Example: "hello"
<code>bool</code>	Boolean value: true or false
<code>list</code>	Ordered collection of values. Example: [1, 2, 3]

Escape sequences inside strings:

Keyword	Description
---------	-------------

<code>\n</code>	Newline
<code>\t</code>	Tab
<code>\"</code>	Double quote inside a string

5. Variables

Variables must be declared with a type keyword and initialized at declaration time. Re-declaring with the same name in the same scope creates a new local variable.

```
int x = 5
float y = 3.14
string name = "PopScript"
bool flag = true
list items = [1, 2, 3]
list words = ["apple", "banana", "cherry"]
```

Reassignment

To update an existing variable use the same type keyword and name. This overwrites the value in the current or nearest parent scope.

```
int count = 0
int count = count + 1    $/ count is now 1
```

List indexing

```
list nums = [10, 20, 30]
int first = nums[0]    $/ 10
int last = nums[2]     $/ 30
```

6. Operators

Arithmetic

Keyword	Description
+	Addition (int, float, string concatenation)
-	Subtraction
*	Multiplication
/	Division (integer division for int, float for float)
%	Modulo (int only)

Comparison

Keyword	Description
==	Equal
!=	Not equal

<	Less than
<=	Less than or equal
>	Greater than
>=	Greater than or equal

Logical

Keyword	Description
and	Logical AND — both sides must be bool
or	Logical OR — both sides must be bool
not	Logical NOT — operand must be bool

```
bool a = true
bool b = false
bool c = a and b    $/ false
bool d = a or b     $/ true
bool e = not a      $/ false

int x = 10 % 3      $/ 1
string s = "hello" + " world" $/ "hello world"
```

7. Control Flow — if

Conditional blocks use `if` and are terminated with `stop`; . There is no `else` — use a second `if` with the opposite condition.

```
int x = 42

if x > 50;
    print("high")
stop;

if x <= 50;
    print("low or equal")
stop;

$/ compound conditions
bool flag = true
if x > 10 and flag == true;
    print("both conditions met")
stop;
```

8. Loops — when, for

when — while loop

`when` repeats its body as long as the condition is true. Use `break` to exit early. Maximum 10,000,000 iterations — exceeding this raises a runtime error.

```
int count = 0
when count < 5;
    print(count)
    int count = count + 1
stop;

$/ with break
int n = 0
when true == true;
    if n >= 3;
        break
    stop;
    int n = n + 1
stop;
```

for...with — foreach loop

Iterates over each element of a list.


```
list fruits = ["apple", "banana", "cherry"]
for fruit with fruits;
    print(fruit)
stop;
```

for...with num() — counted loop

num(N) generates a range from 0 to N-1.

```
for i with num(5);
    print(i)    $/ prints 0 1 2 3 4
stop;
```

9. Functions

Functions are declared with `func` and terminated with `stop`; . Parameters are listed by name only — no type annotations on parameters. Functions can return a value with `return`.

```
func add(a, b);
    int result = a + b
    return result
stop;

int sum = add(3, 4)    $/ 7
print(sum)
```

Exported functions in .plib files

Library files use `export func` to make functions available to importing programs.

```
$/ mylib.plib
export func greet(name);
    print("Hello" name)
stop;

export func add(a, b);
    return a + b
stop;
```

Scope

Each function creates its own scope. Variables declared inside a function are local. Variables from the outer scope are readable but a new declaration with the same name creates a local variable that does not affect the outer one.

```
int x = 10

func test();
    int x = 99    $/ local x, does not change outer x
    print(x)      $/ 99
stop;

test()
print(x)          $/ 10
```

10. Built-in Functions

Keyword	Description
<code>print(v1 v2 ...)</code>	Print one or more values separated by spaces, followed by newline
<code>inputw(prompt?)</code>	Wait for user input from terminal. Returns string. Optional prompt string.
<code>num(N)</code>	Used in for loops. Generates range 0 to N-1.

```
print("hello" "world")    $/ hello world
print(42)                 $/ 42

string name = inputw("Enter your name: ")
print("Hello" name)

for i with num(3);
    print(i)    $/ 0, 1, 2
stop;
```

11. Library System

Importing

```
lib import random        $/ import whole module
lib import random.number  $/ import specific symbol

$/ after importing the module:
int x = random.number(from=0, to=100)

$/ after importing the symbol:
int x = number(from=0, to=100)
```

Search paths for user libraries

When you write `lib import mylib`, the interpreter searches:

1. `./mylib.plib` — same directory as the running script
2. `./libs/mylib.plib` — libs subfolder

Installing packages

```
pop get random
pop get ui
pop list    $/ show installed packages
```

Packages are downloaded from `github.com/Pop-Inc/Pop-Inc.github.io/tree/main/libs` and saved to `./libs/`.

12. Built-in Libraries

random

```
lib import random
lib import random.number

int x = random.number(from=0, to=100)
int y = number(from=1, to=6)    $/ dice roll
```

Keyword	Description
random.number(from, to)	Returns a random int between from and to inclusive

file

```
lib import file

bool ok = file.write(path="log.txt", content="hello")
string data = file.read(path="log.txt")
file.print(path="log.txt")
file.append(path="log.txt", content=" world")
bool exists = file.exists(path="log.txt")
file.delete(path="log.txt")
```

Keyword	Description
file.read(path)	Read file contents, returns string
file.write(path, content)	Write string to file, returns bool
file.append(path, content)	Append string to file, returns bool
file.delete(path)	Delete file, returns bool
file.exists(path)	Check if file exists, returns bool
file.print(path)	Print file contents to terminal

time

```
lib import time

string now = time.now()    $/ "2024-01-15 14:30:00"
string date = time.date()  $/ "2024-01-15"
string clock = time.clock() $/ "14:30:00"
int unix = time.unix()     $/ unix timestamp
time.sleep(s=1)           $/ sleep 1 second
time.sleep(ms=500)        $/ sleep 500 milliseconds
```

Keyword	Description
<code>time.now()</code>	Current date and time as string
<code>time.date()</code>	Current date only
<code>time.clock()</code>	Current time only
<code>time.unix()</code>	Unix timestamp as int
<code>time.sleep(s=N)</code>	Sleep N seconds
<code>time.sleep(ms=N)</code>	Sleep N milliseconds

ui

The `ui` library creates native GUI windows. Call `ui.window()` first, then add widgets, then call `ui.run()` last.

```
lib import ui

func on_click();
    string val = ui.get(name="input1")
    ui.set(name="label1", value="You typed: " + val)
stop;

ui.window(title="My App", width=400, height=300)
ui.label(name="label1", text="Enter something:")
ui.input(name="input1", placeholder="type here...")
ui.button(text="Submit", action="on_click")
ui.space()
ui.checkbox(text="Enable feature", action="on_toggle")
ui.run()
```

Keyword	Description
<code>ui.window(title, width, height)</code>	Create the main window
<code>ui.label(text, name?)</code>	Text label. name makes it updatable via <code>ui.set</code>
<code>ui.button(text, action, name?)</code>	Button. action is a func name string
<code>ui.input(name, placeholder, textinplaceholder?)</code>	Text input field
<code>ui.checkbox(text, action)</code>	Checkbox. action receives "true"/"false"
<code>ui.get(name)</code>	Get current text value of a named widget
<code>ui.set(name, value)</code>	Update text of a named widget
<code>ui.space()</code>	Empty spacer between widgets
<code>ui.run()</code>	Start the event loop — must be last call

13. Memory Management

By default PopScript manages memory automatically. Set `armemory=true` on the first line to switch to manual mode.

Automatic mode (default)

```
armemory=false

int x = 10
$/ x is freed automatically when it goes out of scope
```

Manual mode

```
armemory=true

alloc(buffer, 100)    $/ allocate list of 100 empty slots
int ptr = ref(buffer) $/ get reference to buffer
int val = deref(ptr)  $/ read value from reference
free(buffer)          $/ release memory manually
```

Keyword	Description
<code>alloc(name, size)</code>	Allocate a list of N empty slots
<code>free(name)</code>	Delete a variable and release its memory
<code>ref(name)</code>	Get a reference (copy of value) of a variable
<code>deref(name)</code>	Read value through a reference

14. CLI Reference

Keyword	Description
<code>pop run file.pscript</code>	Run a PopScript file
<code>pop build file.pscript</code>	Embed script into a standalone binary
<code>pop get package</code>	Download and install a package from Pop-Inc repo
<code>pop list</code>	Show all installed packages in <code>./libs/</code>
<code>pop init</code>	Initialize a new PopScript project

pop build

Embeds the `.pscript` source into the `pop` binary itself, producing a self-contained executable. The output file has no extension on Linux or `.exe` on Windows. No interpreter required to run it.

```
pop build mygame.pscript
./mygame           $/ runs directly, no pop needed
```

pop get

Downloads `name.plib` from `github.com/Pop-Inc/Pop-Inc.github.io/tree/main/libs` and saves it to `./libs/name.plib`. Requires internet connection.

```
pop get random
pop get ui
```


15. Examples

FizzBuzz

```
for i with num(100);
  bool fizz = i % 3 == 0
  bool buzz = i % 5 == 0
  if fizz and buzz;
    print("FizzBuzz")
  stop;
  if fizz and not buzz;
    print("Fizz")
  stop;
  if buzz and not fizz;
    print("Buzz")
  stop;
  if not fizz and not buzz;
    print(i)
  stop;
stop;
```

File logger

```
lib import time
lib import file

func log(msg);
  string ts = time.now()
  string line = "[" + ts + "] " + msg + "\n"
  file.append(path="app.log", content=line)
stop;

log("Application started")
log("Processing data")
file.print(path="app.log")
```

Random number guessing game (terminal)

```
lib import random
lib import random.number

int secret = number(from=1, to=100)
int tries = 0
bool won = false

when won == false;
  string guess_str = inputw("Guess (1-100): ")
```

```
    int tries = tries + 1
    $/ note: string to int comparison not yet supported
    $/ use numeric inputw result directly
    print("You guessed:" guess_str)
    print("Tries so far:" tries)
stop;
```

Simple GUI counter

```
lib import ui

int count = 0

func increment();
    int count = count + 1
    ui.set(name="counter", value=count)
stop;

func decrement();
    int count = count - 1
    ui.set(name="counter", value=count)
stop;

ui.window(title="Counter", width=300, height=200)
ui.label(name="counter", text="0")
ui.button(text="+1", action="increment")
ui.button(text="-1", action="decrement")
ui.run()
```

PopScript

pop-inc.github.io · github.com/Pop-Inc · pop.inc.real@gmail.com